

Claude Code Prompt — Implement the BillieBot MVP Software Package

Copy everything below the line into Claude Code (or save it as `PROMPT.md` in the repo root and tell Claude Code to read and execute it).

Mission

You are implementing the complete ROS 2 software package for **BillieBot**, an autonomous indoor robot that patrols an apartment, finds a miniature dachshund named Billie, observes her from a standoff distance, classifies her behavioral state (SLEEPING, RESTING, ACTIVE, BARKING, NOT_FOUND), logs state events to SQLite, and generates a daily activity summary. The MVP is **observe-and-report only** — no engagement behaviors are executed — but the architecture must include the extension points described below so a future Behavior AI (contextual bandit) can be added without interface changes.

Work incrementally, keep every commit buildable (`colcon build` + `colcon test` clean), and prefer boring, well-documented open-source components over clever custom code.

Step 0 — Read the reference material FIRST (do not skip)

Before writing any code, read and summarize back to me:

1. `reference_my_bot/description/` — representative URDF/xacro files describing BillieBot's approximate geometry (chassis, wheel radius/separation, sensor mount frames). **Do not invent geometry.** Derive all kinematic and TF parameters (wheel radius, wheel separation, sensor offsets, `base_link` → `laser`/`camera`/`imu` transforms) from these files. Where a value is missing, add a clearly-marked `# TODO(measure)` parameter with a sensible placeholder and list it in `docs/MEASURE_ME.md`.
2. `reference_my_bot/diff-drive-motor-controller/` — contains (a) the Arduino Nano motor-controller firmware and (b) `diff_drive_base.py`, an existing ROS node that interfaces with that firmware over serial. **This node is the base interface — adapt and wrap it; do not replace it** with `ros2_control`/`diffdrive_arduino` unless you find it fundamentally broken (if so, stop and explain before changing course). Understand its serial protocol, the topics it subscribes/publishes, its units and sign conventions, and whether it already publishes `/odom` and TF. Fill any gaps (e.g., odom integration, IMU passthrough, battery voltage, heartbeat/watchdog behavior) by extending it minimally, preserving protocol compatibility with the firmware.

After reading both, present a short reconciliation note: what the reference code already provides vs. what the architecture below assumes, and your plan to bridge any differences. Wait for my confirmation before scaffolding the workspace.

Target environment

- ROS 2 **Humble**, Python 3.10, C++17 where needed (BT nodes).
- Runtime hosts: Jetson Orin Nano (lidar, OAK-D, Nav2, mission BT, base node) and Raspberry Pi 5 (audio, thermal, NoIR camera, state fusion, logging, reporting). Use **CycloneDDS** with a static peers config (`config/cyclonedds.xml`); Wi-Fi multicast is unreliable.
- Development machine may have **no hardware attached**: every hardware driver node must have a `mock:=true` launch argument that swaps in a simulated publisher (fake `/scan`), fake detections, fake audio events, replayed bag data) so launch files, tests, and the mission logic can be exercised on a desktop.
- Hardware set: RPLidar A1 (USB), Luxonis OAK-D Lite (USB3, runs YOLOv8n on-device via DepthAI spatial detection → 3-D dog position), MLX90640 thermal (I²C, 32×24), Pi Camera 3 NoIR (CSI), ReSpeaker XVF3800 USB mic array (audio + DoA), MAX98357A I²S speaker, Arduino Nano base (via `diff_drive_base.py`), BNO055 IMU (data arrives through the Arduino serial stream — confirm against the reference firmware).

Workspace layout to create

```
billiebot_ws/src/  
  billiebot_interfaces/      # msgs: DogState, DogDetection3D, AudioEvent,  
  ThermalBlob,              # BatteryStatus; actions: ApproachDog, Retreat,  
                             Speak,  
                             # DispenseTreat (stub), all with uniform  
  goal/feedback/result shape  
  billiebot_description/    # URDF/xacro adapted from  
  reference_my_bot/description,  
                             # robot_state_publisher launch  
  billiebot_base/           # wrapped/extended diff_drive_base.py, EKF  
  (robot_localization)  
                             # config fusing wheel odom + IMU, battery monitor  
  node,  
                             # software e-stop service  
  billiebot_navigation/    # slam_toolbox config, AMCL config, Nav2 params  
  (DWB or MPPI),  
                             # keepout/speed-filter masks, waypoint/patrol
```

```

definitions (YAML),

  billiebot_perception/
spatial YOLOv8n,

map frame,

thermal_node +

  billiebot_audio/
stream +

  billiebot_cognition/
→ DogState

room, img_path,

aggregates sleep/

timeline plot,

:8080)

  billiebot_mission/
INVESTIGATE(DoA)

behaviors,

implementation,

1.0 m standoff

rate-limited;

limits are

future policy

  billiebot_bringup/
cyclonedds.xml,

# maps/ directory
# oakd_dog_detector (depthai-ros pipeline,
# publishes DogDetection3D), dog_locator (TF to
# publishes /dog/pose_map + /dog/found),
# thermal_blob, noir_cam node
# audio_classifier (YAMNet TFLite on XVF3800
# DoA readout → AudioEvent), speaker_node (rate-
limited)
# state_fusion (10 s sliding window + hysteresis
# with context[] feature vector and stress_proxy
field),
# dog_logger (SQLite WAL: t, state, conf, x, y,
# context_json, action, outcome; snapshot JPEGs),
# daily_report (systemd-timer-friendly CLI:
# rest/active durations, bark count+times,
# room histogram, snapshots → Markdown+HTML),
# report_server (FastAPI, serves latest report on
# BehaviorTree.CPP mission tree: PATROL ↔
# ↔ TRACK_OBSERVE, stuck-recovery via Nav2
# PolicyDecision BT node (extension point) with
# ObserveOnlyPolicy plugin as the MVP
# engage_actions servers (ApproachDog enforces
# floor and 0.15 m/s cap near dog; Retreat; Speak
rate-limited;
# DispenseTreat returns NOT_IMPLEMENTED) – safety
# enforced inside the action servers, below any
future policy
# incremental + full launch files (below),

```

```
        # host-split launch (jetson.launch.py /
pi.launch.py)
    billiebot_tests/        # discrete verification test scripts (below) +
pytest units
docs/
    VERIFICATION.md        # the incremental bringup ladder with expected
outputs
    MEASURE_ME.md          # physical values still to be measured
```

Deliverable 1 — Incremental launch scripts with verification steps

Create **one launch file per bringup rung** in `billiebot_bringup/launch/`, each with a matching executable check script in `billiebot_bringup/scripts/` (bash or Python) that returns exit code 0/1 so the ladder can be automated. Document the whole ladder in `docs/VERIFICATION.md`. Every rung supports `mock:=true`.

#	Launch file	Starts	Verify script checks
01	<code>01_lidar.launch.py</code>	rplidar driver	<code>/scan</code> publishing ≥ 5 Hz, ranges sane (0.15-12 m)
02	<code>02_base.launch.py</code>	diff_drive_base node + robot_state_publisher	<code>/cmd_vel</code> subscribed; <code>/odom</code> , <code>/tf</code> (odom→base_link), <code>/imu/data</code> , <code>/battery_state</code> publishing; commanded 0.1 m/s produces positive odom x
03	<code>03_ekf.launch.py</code>	robot_localization EKF	<code>/odometry/filtered</code> publishing; TF tree odom→base_link consistent
04	<code>04_slam.launch.py</code>	slam_toolbox (mapping) on top of 01-03	<code>/map</code> publishing and growing; TF map→odom present
05	<code>05_amcl.launch.py</code>	map_server + AMCL with saved map	<code>/amcl_pose</code> publishing; covariance converges below threshold after a short teleop wiggle
06	<code>06_nav2.launch.py</code>	full Nav2 stack	<code>NavigateToPose</code> action available; lifecycle nodes all <code>active</code> ; a test goal 1 m ahead succeeds in mock/sim
07	<code>07_oakd.launch.py</code>	depthai pipeline	RGB preview and depth topics publishing; <code>/dog/detections_3d</code> topic exists; with mock, a fake dog detection streams
08	<code>08_dog_locator.launch.py</code>	dog_locator	<code>/dog/pose_map</code> in map frame when detection + TF present; <code>/dog/found</code> latches
09	<code>09_thermal.launch.py</code>	MLX90640 + blob node	<code>/thermal/image</code> 32×24 at ~4- 8 Hz; <code>/thermal/blob</code> fires on

#	Launch file	Starts	Verify script checks
			warm target (mock: synthetic blob)
10	<code>10_noir.launch.py</code>	NoIR camera node	<code>/noir/image</code> publishing at configured rate
11	<code>11_audio.launch.py</code>	audio_classifier	<code>/audio/events</code> fires on a played bark WAV (test asset in <code>billiebot_tests/assets/</code>); DoA field populated (mock on desktop)
12	<code>12_cognition.launch.py</code>	state_fusion + dog_logger + report_server	<code>/billie/state</code> publishing; SQLite row written on synthetic state change; <code>curl :8080/health</code> OK
13	<code>13_mission.launch.py</code>	mission BT + engage_actions	BT ticks; mode topic cycles PATROL under mock inputs; injected mock AudioEvent triggers INVESTIGATE; mock detection triggers TRACK_OBSERVE and a standoff NavigateToPose goal
14	<code>full_bringup.launch.py</code>	everything, host-split variants <code>jetson.launch.py</code> + <code>pi.launch.py</code>	composite of all checks above

Deliverable 2 — Discrete verification test scripts (Section 6 test plan)

Create one **discrete, runnable script per test case** in `billiebot_tests/scripts/`, named after its TC ID. Every script MUST begin with a structured comment block:

```
# =====
# TEST: TC-XXX-N - <title>
# REQUIREMENT: SYS-XXX-N - <requirement text, abbreviated>
# SETUP: <hardware/space/props needed, launch files to have running>
# PROCEDURE: <numbered steps a human follows; which parts are automated>
# PASS CRITERIA: <objective threshold(s)>
# NOTES: <safety notes, mock-mode behavior>
# =====
```

Automate everything measurable (topic rates, latencies, distances from TF, DB queries); where a human must act (place the dog, carry the robot, play a sound), the script prompts step-by-step, records the result, and writes a JSON result line to `billiebot_tests/results/`. Implement at minimum:

- **TC-PLT-5** (`tc_plt_5_estop_heartbeat.py`) — software e-stop stops `/cmd_vel` effects < 200 ms; kill the base node's heartbeat and confirm firmware stops motors within 500 ms (with hardware; mock asserts the watchdog logic in software).
- **TC-NAV-2** (`tc_nav_2_relocalization.py`) — carry-and-relocalize: AMCL pose error ≤ 0.15 m vs. surveyed reference points (operator enters ground truth when prompted).
- **TC-NAV-3** (`tc_nav_3_dynamic_obstacle.py`) — send goal, walk into the path; assert replan and goal success without contact.
- **TC-NAV-4** (`tc_nav_4_stuck_recovery.py`) — block the robot; assert stuck detection ≤ 5 s of no progress, recovery sequence executes, and SAFE + alert after 3 failures.
- **TC-NAV-5** (`tc_nav_5_speed_limits.py`) — assert `/cmd_vel` ≤ 0.3 m/s in transit and ≤ 0.15 m/s whenever a dog detection is within 2 m (drive past a mock detection).
- **TC-NAV-6** (`tc_nav_6_patrol.py`) — run the patrol waypoint list N laps; assert all waypoints reached, zero interventions.
- **TC-FND-2** (`tc_fnd_2_doa_reprioritize.py`) — inject/play a bark off-route; assert the search queue re-sorts toward the DoA bearing.
- **TC-FND-3** (`tc_fnd_3_standoff.py`) — with a detection present, assert commanded standoff pose keeps 1.0–2.0 m range and the robot never closes inside 1.0 m (TF distance log).
- **TC-FND-4** (`tc_fnd_4_find_time.py`) — 10-trial harness: operator places dog (or mock spawns detection at random waypoint); measure time-to-found; pass if $\geq 8/10 \leq 10$ min.
- **TC-PER-1** (`tc_per_1_rgb_detection.py`) — replay/annotated dataset (or live dog at marked ranges): detection rate ≥ 5 FPS, recall $\geq 85\%$ at ≤ 4 m.
- **TC-PER-2** (`tc_per_2_depth_accuracy.py`) — dog (or dog-sized stuffed target) at taped 1/2/3 m marks; assert ± 0.2 m at 2 m.

- **TC-PER-3** (`tc_per_3_thermal.py`) — warm target at ≤ 1.5 m in darkness; assert blob detection; verify no blob on a cold decoy.
- **TC-PER-4** (`tc_per_4_audio.py`) — play the recorded Billie bark/whine test set + negatives (TV, doorbell, speech); recall ≥ 80 %, report per-class confusion and DoA error $\leq 15^\circ$.
- **TC-PER-5** (`tc_per_5_lowlight.py`) — NoIR detection demo below lux threshold (documents IR-illuminator dependency if it fails).
- **TC-STL-1** (`tc_stl_1_state_accuracy.py`) — replay a 1 h annotated session (bag); compare (`/billie/state`) against labels; report per-state accuracy and transition lag.
- **TC-STL-2** (`tc_stl_2_logging.py`) — inject synthetic state changes; assert one row per transition with all fields (incl. (`context.json`), snapshot path exists).
- **TC-STL-4** (`tc_stl_4_powerloss.py`) — SIGKILL the logger mid-write burst; reopen DB; assert WAL recovery, no corruption, last committed rows intact.
- **TC-RPT-1** (`tc_rpt_1_daily_report.py`) — seed a synthetic day of events; run (`daily_report`); assert totals, bark count/times, timeline image, and snapshots all present and arithmetically correct.
- **TC-RPT-2** (`tc_rpt_2_report_access.py`) — fetch the report over HTTP from another host on the LAN; assert 200 + content hash matches.
- **TC-PLT-1** (`tc_plt_1_endurance.py`) — logged patrol soak; pass at ≥ 60 min with battery telemetry curve saved.
- **TC-PLT-2** (`tc_plt_2_battery_safe.py`) — feed descending mock battery voltage; assert SAFE mode (motion stop + alert) at 3.5 V/cell.

Also provide (`run_all_mock_tests.sh`) (everything runnable without hardware) and wire the pure-software ones into (`colcon test`).

Extension points that MUST exist (future Behavior AI — do not implement the AI)

1. (`PolicyDecision`) BT node with a pluginlib-style interface (`decide(context) → {OBSERVE, APPROACH, RETREAT, SPEAK, TREAT}`); MVP ships only (`ObserveOnlyPolicy`) (always OBSERVE). Selecting the policy is a launch parameter.
2. All five actions exist as action servers with a uniform interface even though the MVP tree only ever uses navigation-internal approach/retreat.
3. (`DogState.msg`) includes (`float32[] context`) and (`float32 stress_proxy`) (MVP heuristic: bark-rate + rapid-retreat motion), populated on every publish.
4. (`dog_logger`) records (`(context, action, outcome)`) for every action executed near the dog, including OBSERVE no-ops at a low sample rate.
5. Standoff floor, near-dog speed cap, and speaker rate limit are enforced inside the action servers/controllers, never in the policy layer.

Working agreements

- Explain your plan before each package; build and run the relevant rung's verify script before moving to the next rung.
- All tunables in YAML under each package's `config/`; no magic numbers in code. Kinematics come from the URDF, not hardcoded.
- Type hints, docstrings, and a README per package (purpose, nodes, topics, params, launch usage).
- If the reference `diff_drive_base.py` or firmware conflicts with anything above (topic names, units, watchdog behavior), STOP and present options rather than silently diverging.
- Never command motion in tests without an operator-confirm prompt when `mock:=false`.

Begin with Step 0.